A THEOREM PROOFS

A.1 Proof of Theorem 3.2

Proof. We prove the two cases separately.

Case 1: $\alpha_i > 0$ for all i. Consider any object $v \notin \mathrm{KSky}(\mathcal{D}, o, K)$. By the definition of K-skyband, v is dominated by at least K distinct objects. That is, there exist K objects u such that $f_i(u) \le f_i(v)$ for all i, and for at least one coordinate the inequality is strict.

Since all $\alpha_i > 0$, multiplying each inequality by α_i and summing them preserves the strict inequality:

$$\sum_{i=1}^{m} \alpha_i f_i(u) < \sum_{i=1}^{m} \alpha_i f_i(v).$$

Thus, all these K objects have strictly smaller weighted distance than v. Hence v cannot rank within the top-κ for any $\kappa < K$. Therefore, every top-κ nearest neighbor must lie in $\mathrm{KSky}(\mathcal{D}, o, K)$.

Case 2: Some $\alpha_i = 0$. Let T be any top-κ nearest-neighbor set of o under the weighted distance. If $T \subseteq \mathrm{KSky}(\mathcal{D}, o, K)$, we are done.

Otherwise, choose an object $v \in T$ that is not in $\mathrm{KSky}(\mathcal{D}, o, K)$. Then v is dominated by at least K objects. For each such dominator u, we have $f_i(u) \le f_i(v)$ for all i. Since $\alpha_i \ge 0$, it follows that

$$\sum_{i=1}^{m} \alpha_i f_i(u) \le \sum_{i=1}^{m} \alpha_i f_i(v).$$

Because $|T| = \kappa < K$, at least one dominator u is not in T. We may replace v in T with such a u without increasing the weighted distance. If the new element still lies outside $\mathrm{KSky}(\mathcal{D}, o, K)$, we repeat the same replacement process. Since the dataset is finite, this process must terminate. Eventually we obtain a top-κ set entirely contained in $\mathrm{KSky}(\mathcal{D}, o, K)$. □

A.2 Proof of Theorem 3.3

Proof. We prove that every object in MLSky belongs to the K-skyband of CS.

Recall that L_i is defined as the skyline (i.e., 1-skyband) of the residual set $CS_i = CS \setminus \bigcup_{j=1}^{i-1} L_j$. By the definition of skyline, no object in CS_i dominates any object in L_i. Therefore, if an object $x \in L_i$ is dominated by some object in CS, such a dominator must belong to one of the earlier layers $\bigcup_{j=1}^{i-1} L_j$.

Now consider any object $x \in$ MLSky. Then $x \in L_i$ for some $1 \le i \le \ell$. As argued above, all objects that dominate x must lie in $\bigcup_{j=1}^{i-1} L_j$. Hence, the number of objects in CS that dominate x is at most

$$\left| \bigcup_{j=1}^{i-1} L_j \right|.$$

Since $\sum_{j=1}^{\ell} |L_j| \le K$, we have

$$\left| \bigcup_{j=1}^{i-1} L_j \right| = \sum_{j=1}^{i-1} |L_j| \le \sum_{j=1}^{\ell-1} |L_j| \le K - 1.$$

Therefore, every object in MLSky is dominated by at most $K - 1$ objects in CS. By the definition of the K-skyband, this implies that MLSky is a subset of the K-skyband of CS. □

Algorithm 6: Weight-Aware Search on WTNG

Input: Graph G; query q with weight vector $\boldsymbol{\alpha}$; beam width ef_{search}; entry-point set EP; return size k; template set $\mathcal{T}$; number of templates n_t

Output: Top-k approximate nearest neighbors $\mathcal{R}$

```
Procedure WTNGSearch(G, q, EP, k, T, n_t):
    Min-heap S ← ∅;
    Max-heap R ← ∅;
    VISITED ← ∅;
    foreach s ∈ EP do
        VISITED.add(s);
        d_s ← d_α(q, s);
        S.add(d_s, s);
        R.add(d_s, s);
        if |R| > ef_search then
            pop the farthest object from R;
    T_q ← the n_t templates in T closest to α;
    π ← (i_1, ..., i_m) s.t. α_{i_1} ≥ ··· ≥ α_{i_m};
    while S is not empty do
        (d_v, v) ← extract-min from S;
        if |R| = ef_search then
            (d_u, u) ← R.peek-max();
            τ ← d_u;
            if d_v > τ then
                break;
        NS(v) ← { o ∈ G[v] | ∃ α^(j) ∈ T_q s.t. b_j(v, o) = 1 };
        foreach o ∈ NS(v) and o ∉ VISITED do
            VISITED.add(o);
            if |R| < ef_search then
                d_o ← d_α(q, o);
                S.add(d_o, o);
                R.add(d_o, o);
            else
                d_o ← AggDistEarlyStop(o, q, α, τ, π);
                if d_o < τ then
                    S.add(d_o, o);
                    R.add(d_o, o);
                    if |R| > ef_search then
                        pop the farthest object from R;
    return Top-k elements of R;
```

B WTNG SEARCH ALGORITHM

Algorithm 6 presents the weight-aware beam search procedure on WTNG. Given a query q with weight vector $\boldsymbol{\alpha}$, the algorithm initializes a candidate min-heap $\mathcal{S}$ and a result max-heap $\mathcal{R}$ using the entry-point set EP. All entry points are evaluated by the aggregated distance $d_{\boldsymbol{\alpha}}(q, \cdot)$ and inserted into both heaps.

We then select the n_t templates in $\mathcal{T}$ that are closest to $\boldsymbol{\alpha}$, forming $\mathcal{T}_q$. During traversal, only edges whose bit masks are activated under at least one template in $\mathcal{T}_q$ are explored. This effectively induces a query-adaptive overlay graph.

Algorithm 7: Early-Stopping Distance Evaluation

Input: Object o; query q; weight vector $\boldsymbol{\alpha}$; threshold τ; permutation π
Output: Aggregated distance d_o (or $+\infty$ if early-stopped)

```
Procedure AggDistEarlyStop(o, q, α, τ, π):
    d_o ← 0;
    for t ← 1 to m do
        i ← π[t];
        d_o ← d_o + α_i · d(q.e_i, o.e_i);
        if d_o > τ then
            return +∞;
    return d_o;
```

The beam search proceeds by repeatedly extracting the closest candidate v from $\mathcal{S}$ and expanding its activated neighbors. The behavior differs depending on whether the result heap $\mathcal{R}$ has reached its capacity.

- $|\mathcal{R}| < ef_{\text{search}}$. The beam is not yet full. For each unvisited neighbor o of v, we compute its full aggregated distance $d_{\boldsymbol{\alpha}}(q, o)$ and insert o into both $\mathcal{S}$ and $\mathcal{R}$. No pruning is applied in this phase.
- $|\mathcal{R}| = ef_{\text{search}}$. The result heap is full, and its farthest element defines the current threshold τ. If $d_{\boldsymbol{\alpha}}(q, v) > \tau$, the search terminates since no better candidate can be found. Otherwise, neighbors are evaluated using the early-stopping routine (Algorithm 7), and only candidates with distance smaller than τ are inserted into the heaps.

The process continues until no candidate in $\mathcal{S}$ can improve the current threshold. Finally, the top-k objects in $\mathcal{R}$ are returned.

Early-Stopping Distance Evaluation. Algorithm 7 implements the early-stopping distance evaluation routine. Given a candidate object o, the aggregated distance $d_{\boldsymbol{\alpha}}(q, o)$ is computed incrementally following the permutation π, where feature indices are sorted in non-increasing order of the query weights.

At each iteration, the partial sum d_o accumulates a weighted per-feature distance term. Since all distance components are non-negative, the partial sum is a lower bound of the final aggregated distance. If the partial sum exceeds the current threshold τ, the computation terminates immediately and returns $+\infty$, indicating that o cannot improve the current beam.

Otherwise, once all features have been processed, the full aggregated distance is returned. This mechanism avoids unnecessary distance computations for candidates that are clearly worse than the current threshold.

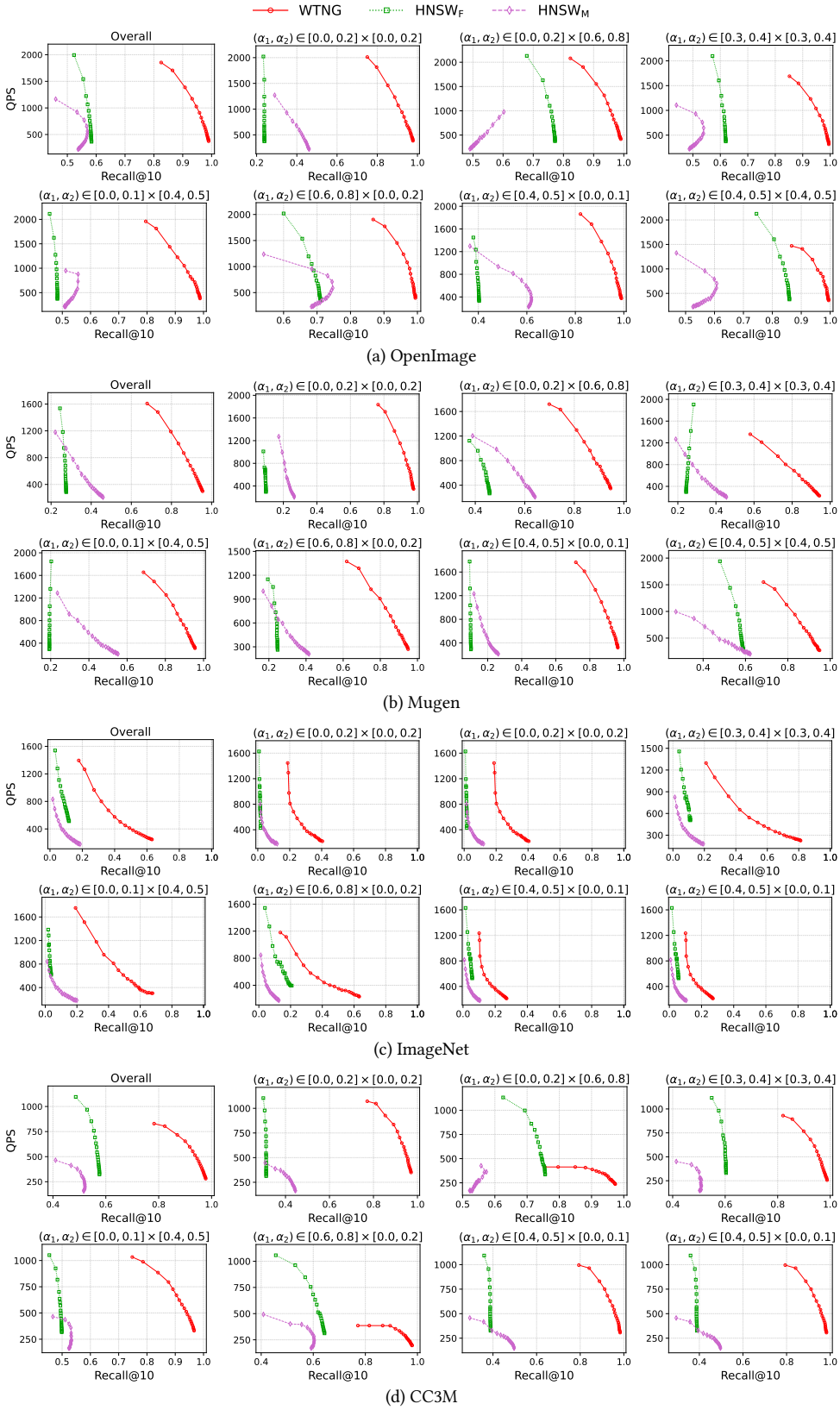
C DATASET DESCRIPTION

A detailed description of our datasets is provided below.

- OpenImage [3, 59]: The OpenImage dataset is an open benchmark connecting vision and language. It comprises around 500K training images and 41K validation images. Each image is paired with localized narratives provided by annotators. To construct the feature representations for each object, images are converted into 768-dimensional vectors using ViT [17], and localized narratives are transformed into 768-dimensional vectors using BERT [16]. We use these two vectors for the two-vector setting. For the three-vector setting, we further translate the localized narratives into Chinese and encode the translated text into an additional 768-dimensional vector using BERT. The query set Q ($|Q| = 1,000$) is randomly selected from the validation set.
- Mugen [4, 33]: The Mugen dataset is a large-scale video-audio-text dataset designed for multimodal understanding and generation tasks. It consists of 592K video sequences paired with audio, textual descriptions, and structured annotations. Audio signals are encoded into 768-dimensional vectors using Wav2Vec2 [9], textual descriptions are encoded into 768-dimensional vectors using BERT, and videos are represented by structured game states extracted from metadata JSON files and encoded into 768-dimensional vectors using a fixed Gaussian random projection, motivated by the Johnson-Lindenstrauss lemma [39]. We use all three vectors for the three-vector setting, while for the two-vector setting, we use only the audio and text vectors. The query set Q ($|Q| = 1,000$) is randomly sampled from the entire dataset, and the remaining vectors are used as the database.
- ImageNet [1, 15]: ImageNet is a large-scale image database organized according to the WordNet hierarchy. We adopt three ImageNet-derived embedding sets consisting of 1.28M vectors: ImageNet-DINOv2, ImageNet-EVA02, and ImageNet-ConvNeXt, which are generated using different embedding models as described in [13]. We use the above three vectors for the three-vector setting, whereas for the two-vector setting, we retain only the ImageNet-DINOv2 and ImageNet-EVA02 vectors. The query set Q ($|Q| = 1,000$) is randomly sampled from the entire dataset, and the remaining vectors are used as the database.
- CC3M [2, 64]: CC3M is a large-scale image-text dataset containing approximately 3.3M pairs. The captions are collected from webpage alt text and then filtered to remove noisy descriptions. Due to invalid or expired image URLs, we restrict our experiments to the 290K image-text pairs that can be successfully downloaded. Images are converted into 768-dimensional vectors using ViT [17], and captions are transformed into 768-dimensional vectors using BERT [16]. We use these two vectors for the two-vector setting. For the three-vector setting, we further translate the captions into Chinese and encode the translated text into an additional 768-dimensional vector using BERT. The query set Q ($|Q| = 1000$) is randomly selected from the validation set.

D THREE-VECTOR MVQ PERFORMANCE

The complete experimental results for the three-vector MVQ setting are shown in Figure 11, corroborating the observations reported in the main text.



(a) OpenImage

(b) Mugen

(c) ImageNet

(d) CC3M

Figure 11: Accuracy-Efficiency Tradeoff Results in Three-Vector Setting (Upper and Right is Better)

E TWO-VECTOR MVQ PERFORMANCE

The complete experimental results for the two-vector MVQ setting are shown in Figure 12, corroborating the observations reported in the main text.

F FULL RESULTS OF ABLATION STUDY AND SENSITIVITY ANALYSIS

The complete results of the ablation studies on the WTNG edge pruning strategy and edge activation mechanism, as well as the sensitivity analysis of M and $ef_{\text{construction}}$, are reported in Figure 13, Figure 14, and Figure 15, respectively, corroborating the observations reported in the main text.

G ADDITIONAL ABLATION STUDY AND SENSITIVITY ANALYSIS

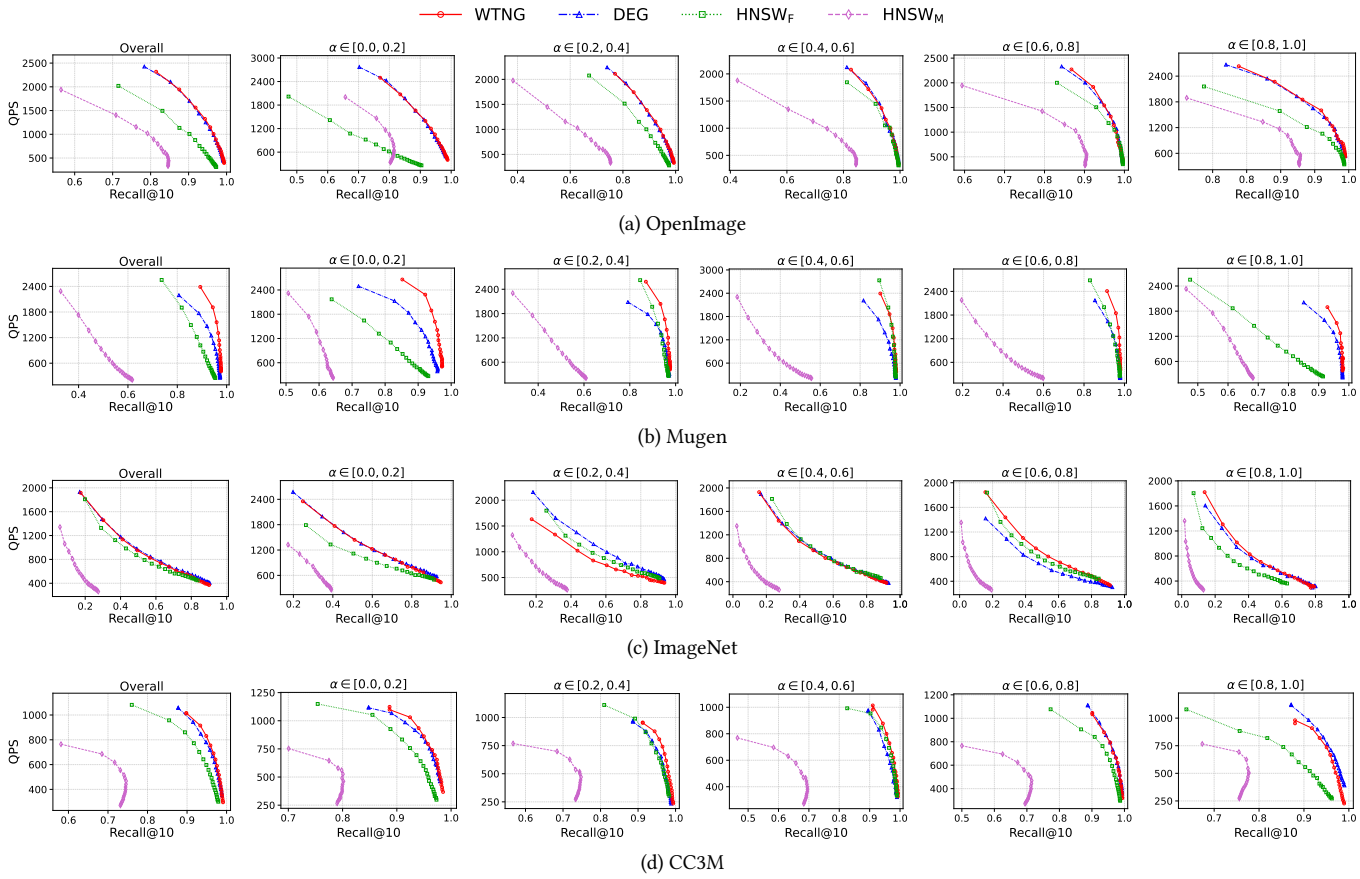
Ablation Study of Candidate Acquisition Method. Recall that in Section 3.2, the K-skyband candidates are approximated via skyline computation. Skyline processing can be carried out either approximately using subspace skyline methods (as adopted by our WTNG), or exactly using classical skyline algorithms. Classical exact methods admit time complexity $O(N \log^{m-2} N)$ [43], where N is the number of data points and $m \geq 3$.

One representative approach is the sort-and-scan method [56], implemented using a binary indexed tree (BIT) [19]. The method begins by sorting all points according to one feature and then scans them in that order. During the scan, a binary indexed tree maintains a compact summary indexed by a second objective, allowing each new point to be tested for dominance via a logarithmic-time prefix query instead of comparing against all prior points.

We replace the subspace skyline module in WTNG with the sort-and-scan method (implemented via binary index tree), denoted as WTNG+BIT. We compare WTNG and WTNG+BIT on the Mugen dataset, with results reported in Figure 16. In general, the two variants achieve comparable performance. However, under certain weight configurations, their performance can differ noticeably, suggesting that the skyline module affects the candidate composition. Compared with the BIT-based exact skyline computation, our approximate subspace skyline may admit additional vertices beyond the exact solution set, increasing candidate diversity and thereby altering the search trajectory for those $\boldsymbol{\alpha}$, while keeping the overall performance broadly similar.

Ablation Study of Early-Stopping Mechanism. To evaluate the effectiveness of the early-stopping mechanism, in which partial distance accumulation is compared against full distances to terminate computation early, we construct a variant of WTNG without this component, denoted as $\text{WTNG}_{\text{w/o stop}}$. Experimental results on the Mugen dataset are shown in Figure 17. Overall, WTNG achieves approximately a 1.2× speedup over $\text{WTNG}_{\text{w/o stop}}$ at 90% recall. This suggests that there often exists a non-trivial gap in the query-to-candidate distances, separating strong candidates from inferior ones. As a result, partial distance accumulation frequently exceeds the current best value, enabling early termination of unnecessary computations. Although $\text{WTNG}_{\text{w/o stop}}$ may perform comparably to WTNG in some cases, WTNG consistently dominates across all settings, with its QPS–Recall@10 curve lying above that of $\text{WTNG}_{\text{w/o stop}}$. These results confirm the effectiveness of the early-stopping mechanism in improving search efficiency.

Sensitivity to the Size of Entry Points. Figure 18 presents the sensitivity of WTNG to the size bound B of the entry-point set. The entry-point set determines the number of initial starting vertices used during search, which directly affects the exploration breadth and search overhead. We choose $B = 12, 15, 18$ and refer to them as WTNG_{12}, WTNG_{15} and WTNG_{18}. We can observe that WTNG_{12} exhibits noticeably worse performance compared to WTNG_{15} and WTNG_{18}, indicating that an insufficient number of entry points limits search coverage and degrades accuracy-efficiency tradeoffs. In contrast, raising the entry-point-set size bound beyond 15 yields only marginal improvements, as WTNG_{15} and WTNG_{18} achieve nearly identical QPS-Recall@10 curves across all reported settings. These results indicate that a moderate entry-point-set size budget (i.e., 15) strikes a favorable balance between exploration diversity and search efficiency.



(a) OpenImage

(b) Mugen

(c) ImageNet

(d) CC3M

Figure 12: Accuracy-Efficiency Tradeoff Results in Two-Vector Setting (Upper and Right is Better)

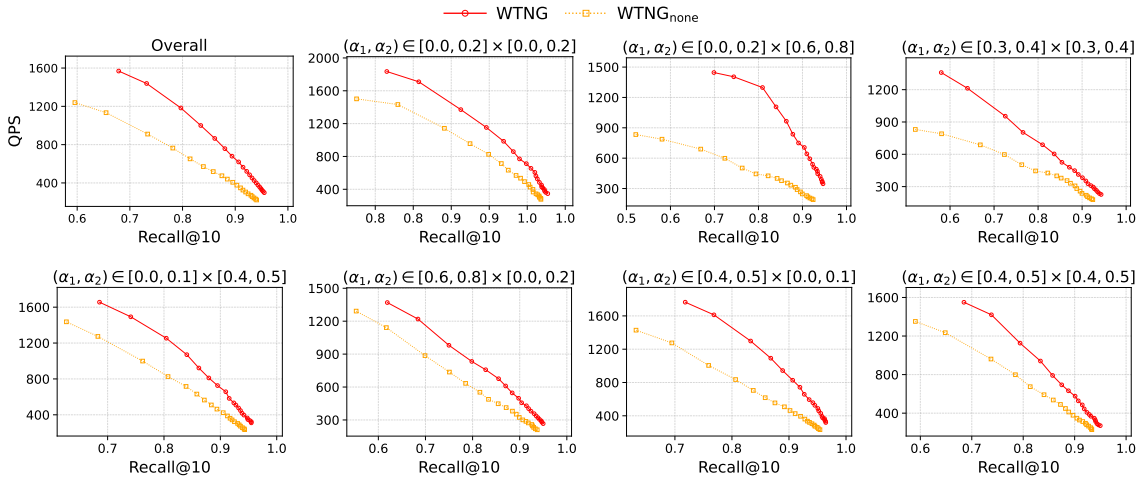


Figure 13: Ablation Study for Edge Prune Strategy

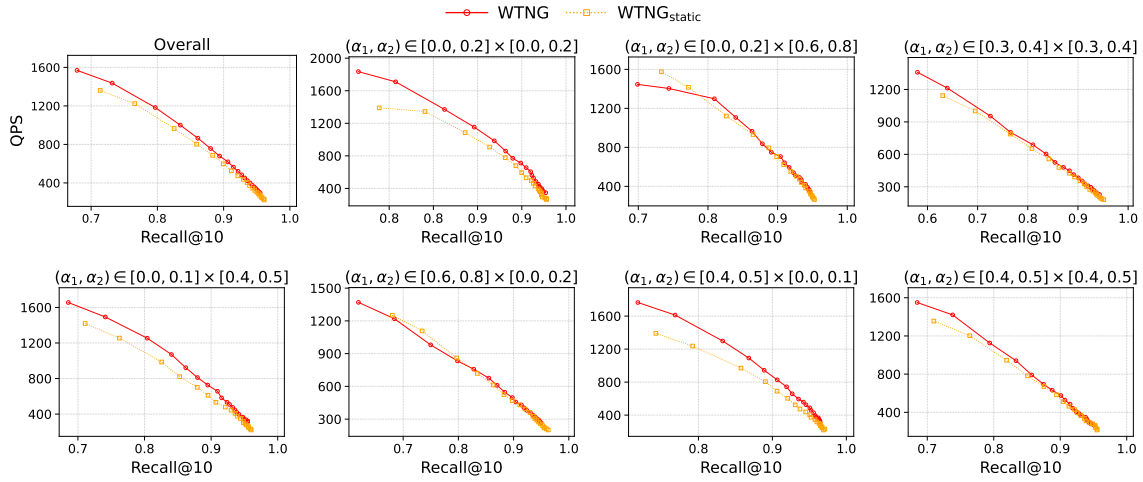
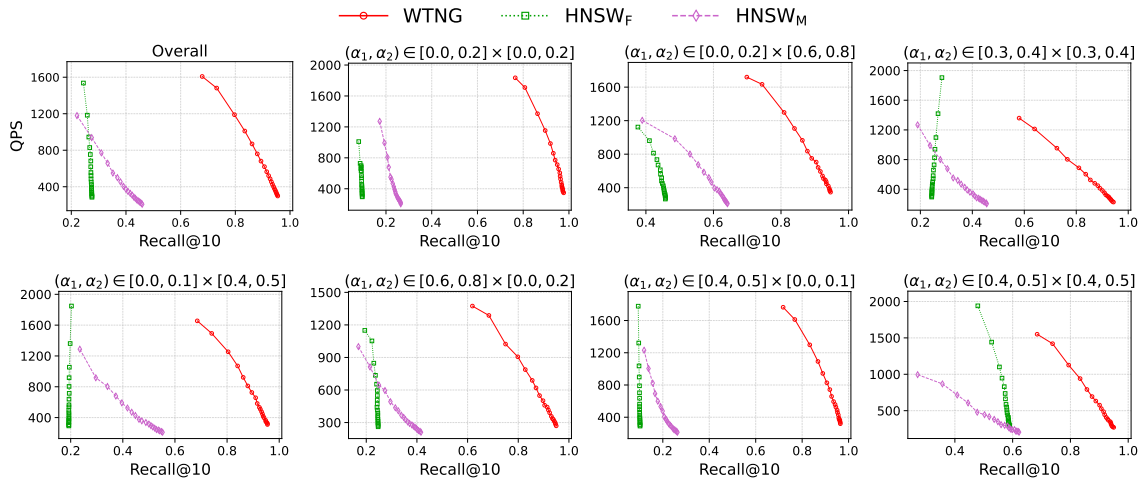
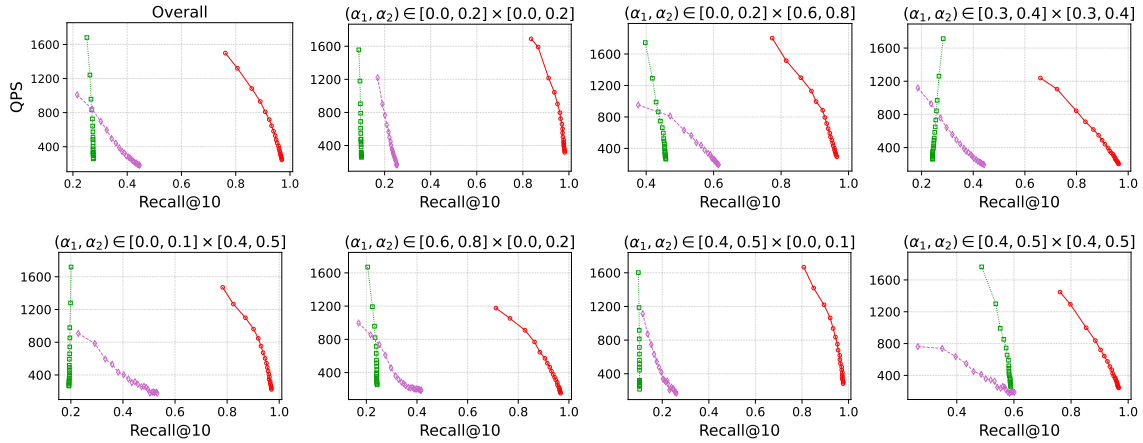


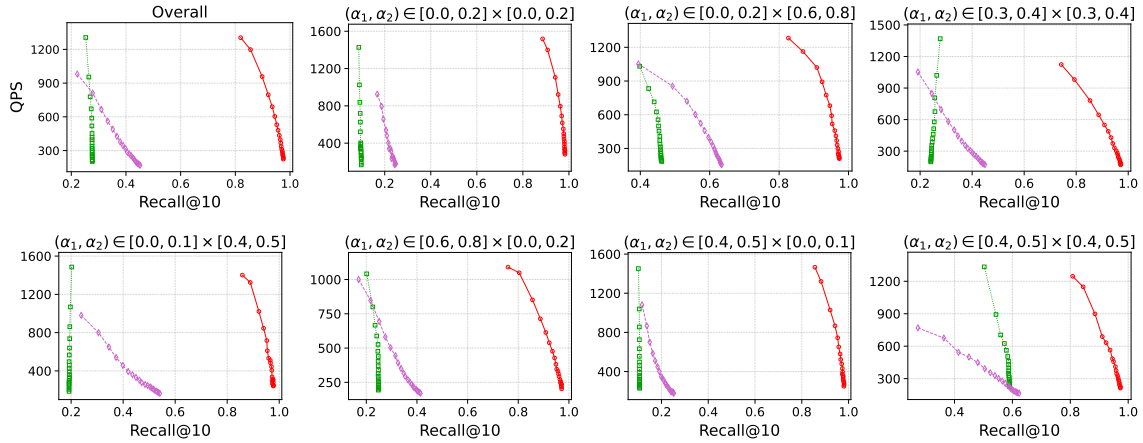
Figure 14: Ablation Study for Edge Activation Mechanism



(a) $M = 40, ef_{\text{construction}} = 200$



(b) $M = 50, ef_{\text{construction}} = 250$



(c) $M = 60, ef_{\text{construction}} = 300$

Figure 15: Performance with Varying M and $ef_{\text{construction}}$

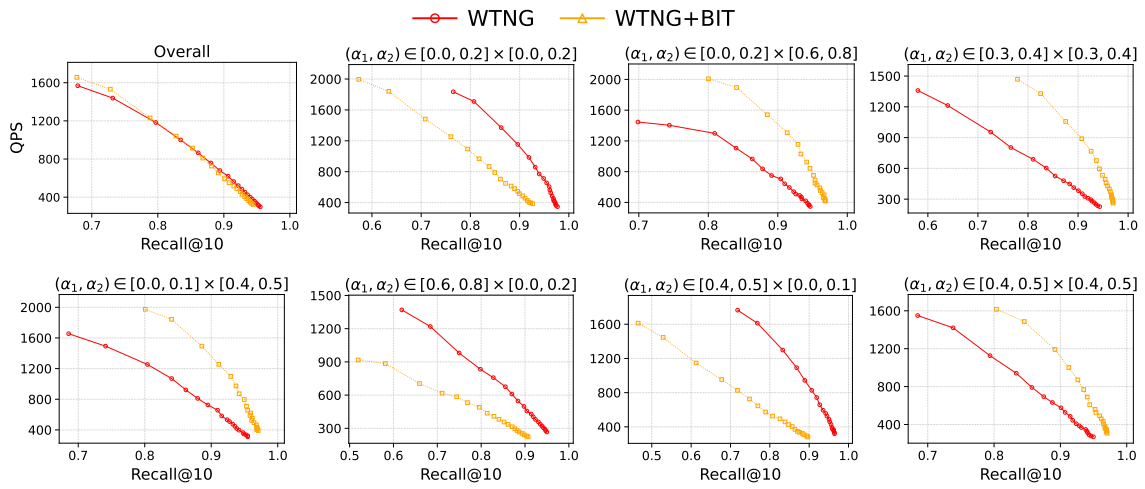


Figure 16: Ablation Study for Candidate Acquisition Method

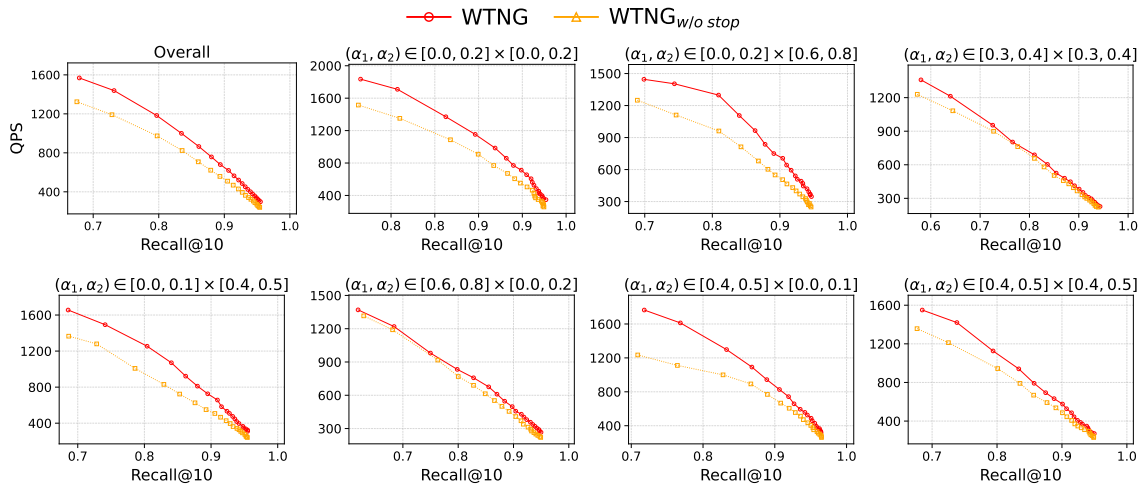


Figure 17: Ablation Study for Early-Stopping Mechanism

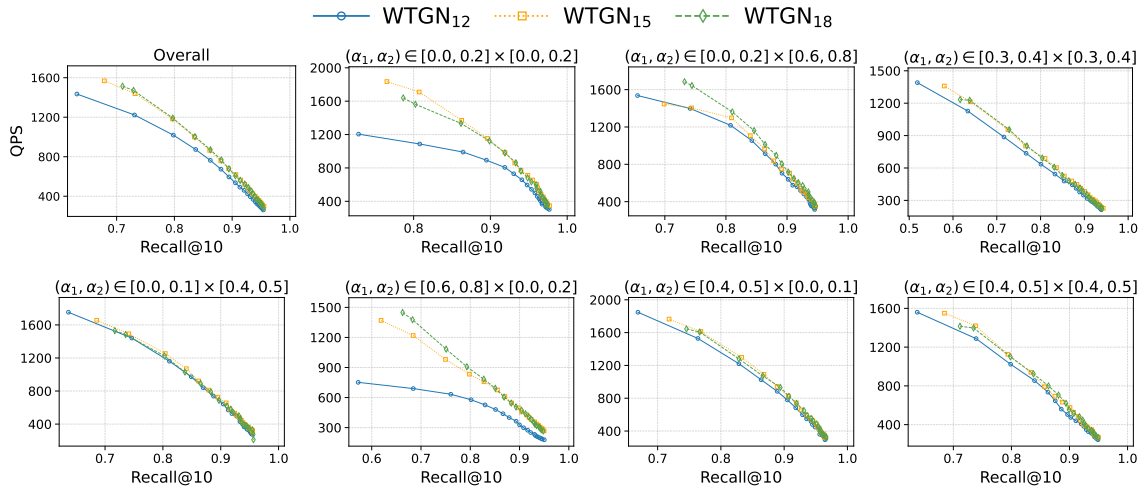


Figure 18: Sensitivity Study of Entry Point Size